



KATHFORD INTERNATIONAL COLLEGE OF ENGINEERING AND
MANAGEMENT
DEPARTMENT OF COMPUTER AND ELECTRONICS COMMUNICATION &
INFORMATION ENGINEERING

A
MAJOR PROJECT REPORT
ON
**REAL-TIME ELDERLY FALL DETECTION SYSTEM USING DEEP
LEARNING**

By:
Bishal Humagain (10/BEI/2021)
Nishant Bhandari (15/BEI/2021)
Sabin Silwal (17/BEI/2021)

Lalitpur, Nepal

Chaitra, 2082



KATHFORD INTERNATIONAL COLLEGE OF ENGINEERING AND
MANAGEMENT
BALKUMARI, LALITPUR

(REAL-TIME ELDERLY FALL DETECTION USING DEEP LEARNING)

[Subject Code: EX 755]

By:

Bishal Humagain (KIC078BEI010)

Nishant Bhandari (KIC078BEI015)

Sabin Silwal (KIC078BEI017)

A PROJECT WAS SUBMITTED TO THE DEPARTMENT OF COMPUTER
AND ELECTRONICS, COMMUNICATION & INFORMATION
ENGINEERING IN PARTIAL FULLFILLMENT OF THE REQUIREMENT
FOR THE BACHELOR'S DEGREE IN ELECTRONICS,
COMMUNICATION & INFORMATION / COMPUTER ENGINEERING

DEPARTMENT OF COMPUTER AND ELECTRONICS, COMMUNICATION &
INFORMATION ENGINEERING
LALITPUR, NEPAL

Chaitra, 2082

KATHFORD INTERNATIONAL COLLEGE OF ENGINEERING AND
MANAGEMENT
BALKUMARI, LALITPUR

DEPARTMENT OF COMPUTER AND ELECTRONICS, COMMUNICATION &
INFORMATION ENGINEERING

The undersigned certify that they have read, and recommended to this department for acceptance, a project report entitled “**Real-Time Elderly Fall Detection Using Deep Learning**” submitted by Bishal Humagain (KIC078BEI010), Nishant Bhandari (KIC078BEI015), Sabin Silwal (KIC078BEI017) in partial fulfilment of the requirements for the Bachelor’s degree in Electronics & Communication / Computer Engineering.

Supervisor, name of Supervisor Title

Name of the Department

External Examiner, name of External Title

Name of the Organization, he/she belongs to

Coordinator, Name of Coordinator Title

Name of the coordinating committee

DATE OF APPROVAL:

COPYRIGHT

The author has agreed that the Library, Department of Computer and Electronics, Communication and Information Engineering, Kathford International College may make this report freely available for inspection. Moreover, the author has agreed that permission for extensive copying of this project report for scholarly purpose may be granted by the supervisors who supervised the project work recorded herein or, in their absence, by the Head of the Department wherein the project report was done. It is understood that the recognition will be given to the author of this report and to the Department of Computer and Electronics, Communication and Information Engineering, Kathford International College in any use of the material of this project report. Copying or publication or the other use of this report for financial gain without approval of to the Department of Computer and Electronics, Communication and Information Engineering, Kathford International College and author's written permission is prohibited.

Request for permission to copy or to make any other use of the material in this report in whole or in part should be addressed to:

Head

Department of Computer and Electronics, Communication and Information
Engineering, Kathford International College of Engineering and Management,
Lalitpur, Nepal

Departmental Acceptance Letter

DEPARTMENTAL ACCEPTANCE

The project entitled **“Real-Time Elderly Fall Detection Using Deep Learning”**, submitted by Bishal Humagain (KIC078BEI010), Nishant Bhandari (KIC078BEI015), Sabin Silwal (KIC078BEI017) in partial fulfilment of the requirements for the Bachelor’s degree in Electronics, Communication & Information / Computer Engineering has been accepted as a bonafide record of work carried out by them in this department.

Chaitya Shova Shakya

HOD

Department of Computer and Electronics, Communication and Information
Engineering, Kathford International College of Engineering and Management,
Lalitpur, Nepal

Acknowledgement

We would like to express our sincere gratitude to all those who have supported and guided us throughout the completion of our minor project, **Real-Time Elderly Fall Detection Using Deep Learning**.

Firstly, we extend our sincere thanks to our supervisor **Er. Bimal Aryal**, for his invaluable guidance, continuous support. His insights and encouragement have been instrumental in shaping and successfully completing this project. We would also like to express our sincere gratitude to our Project Coordinator **Er. Saban Kumar K.C**, for his valuable guidance, coordination, and support throughout the project. His suggestions and supervision have greatly contributed to the smooth execution of our work. We sincerely thank **Krishna Murari Sir** for providing the necessary hardware components, which made the execution of this project possible. Furthermore, we extend our appreciation to all the respected faculty members of the **Department of Computer and Electronics Communication & Information Engineering** for their continuous encouragement, academic guidance, and for providing us with the necessary resources to complete this project successfully.

Finally, we thank everyone who directly or indirectly contributed to the successful completion of this project. This project would not have been possible without the collective efforts of these individuals, and we truly appreciate their time, effort, and expertise.

Bishal Humagain

Nishant Bhandari

Sabin Silwal

Abstract

Falls are one of the leading causes of injury among elderly individuals, and quick assistance after a fall is crucial for reducing serious health risks. Many existing fall detection systems struggle to balance accuracy and efficiency; simple threshold-based methods often produce false alarms, while complex models are difficult to implement on low-power wearable devices. This project presents a real-time wearable fall detection system using a lightweight 1D-Convolutional Neural Network (1D-CNN) trained on the SisFall dataset. The system is implemented using a hardware prototype built with an ESP32 microcontroller and an MPU-6050 sensor, allowing on-device processing of motion data and wireless alert transmission via Wi-Fi. The developed model achieved a validation accuracy of 93%, with 92% recall, 88% precision, and an F1-score of 0.90 for fall detection. The model runs in under 50 ms on the ESP32 using FreeRTOS, meeting the real-time requirements of a wearable safety device. When a fall is detected, the system triggers a local LED/buzzer alert and sends a remote notification via IFTTT. The result suggests that deep learning-based fall detection can be effectively deployed on low-cost microcontroller hardware, providing a practical and reliable solution for improving elderly safety.

Keyword: Fall Detection, Deep Learning, 1D-CNN, Wearable Device, Real-Time Monitoring.

Table of Contents

COPYRIGHT	i
Departmental Acceptance Letter	ii
Acknowledgement	iii
Abstract	iv
Table of Contents	v
List of Figures	vii
List of Tables	viii
List of Abbreviations/Acronyms	ix
1. INTRODUCTION	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Objective	2
1.4 Scope	3
2. LITERATURE REVIEW	4
3. FEASIBILITY STUDY	6
3.1 Technical Feasibility	6
3.2 Economic Feasibility	6
4. PROJECT METHODOLOGY	7
4.1 Block Diagram of the developed System.....	7
4.2 Flowchart of the Developed System	8
4.3 Model Classification	9
5. SYSTEM DESIGN	13
5.1 Schematic Diagram:	13
5.2 Model Development:.....	14
5.2.1 Dataset Analysis.....	14
5.2.2 Data Acquisition and Preprocessing.....	16
5.2.3 Model Trained using 1D CNN.....	18
5.2.4 Model Training on Kaggle GPU Environment.....	19
5.2.5 Quantization and Embedded Deployment.....	21
5.2.6 Evaluation Metrics for Fall detection	22
5.2.6.1 Precision:	22
5.2.6.2 Recall:	22
5.2.6.3 F1 Score:.....	23
6. HARDWARE AND SOFTWARE REQUIREMENT	24
6.1 Hardware Requirement	24

6.2 Software Requirements	24
7. RESULT AND DISCUSSION	25
7.1 Dataset Processing Summary	25
7.2 Model Accuracy and Loss	26
7.3 Confusion Matrix and Classification Report	27
7.4 Prototype Implementation	29
7.5 Alert and Notification	31
8. CONCLUSION	32
9. LIMITATION AND FUTURE WORK	33
10. REFERENCES	34
11. APPENDICES	36

List of Figures

Figure 4.1: Block Diagram	7
Figure 4.2: Flowchart	8
Figure 4.3: Simple 1D convolutional neural network (CNN) architecture.....	10
Figure 5.1: Schematic diagram	13
Figure 5.2.6: Confusion Matrix.....	22
Figure 7.2: Accuracy and Loss Curve of 1D CNN Model.....	26
Figure 7.3: Confusion Matrix of 1D CNN Model.....	27
Figure 7.4: Prototype Implementation.....	29
Figure 7.5: Email Alert.....	31

List of Tables

Table 5.2.1.1: Raw SisFall Data	15
Table 5.2.1.2: Converted SisFall Data	15
Table 5.2.2: Per-channel Z-score normalization statistics	17
Table 5.2.3 1D CNN layer-by-layer architecture	18
Table 5.2.4.1: Model training hyperparameters.....	19
Table 5.2.4.2: Selected epoch training log	20
Table 5.2.5: TFLite INT8 quantization parameters	21
Table 7.1: SisFall dataset processing summary	25
Table 7.3: Full classification report on the 14,882-window validation set	28

List of Abbreviations/Acronyms

- AFD: Automatic Fall Detection
- ADL: Activity of Daily Living
- CNN: Convolutional Neural Network
- GPIO: General-Purpose Input/Output
- HMM: Hidden Markov Model
- IMU: Inertial Measurement Unit
- I2C: Inter-Integrated Circuit
- MCU: Microcontroller Unit
- ROC: Receiver Operating Characteristic
- SVM: Sum Vector Magnitude
- TFLite: TensorFlow Lite

1. INTRODUCTION

1.1 Background

The world is aging at the fastest pace ever recorded. According to a report by World Health Organization, it has projected the number of people age 60 years and above, they will nearly triple between 12 percent in 2015 to 22 percent in 2050. Globally, however, falls became the 2nd leading cause of death through unintentional injury and the most common cause of death through injuries for the elderly over the age of 65 years [1].

Response time is a critical factor in determining the clinical outcome after a fall. When an elderly person experiences a “long lie,” meaning they remain on the floor for more than an hour, it can lead to serious health risks such as dehydration, pressure ulcers, hypothermia, and post-fall anxiety. This anxiety can reduce mobility and increase the likelihood of future falls, creating a vicious cycle. To reduce delays in assistance, AFD systems were developed to detect falls in real time and alert caregivers immediately. These systems are generally categorized into two types: ambient-based systems, which use technologies like cameras, pressure-sensitive floors, or depth sensors, and wearable-based systems that use IMUs consisting of accelerometers and gyroscopes. While ambient systems can monitor environments, they are limited to specific spaces, can be expensive, and often raise privacy concerns. In contrast, wearable IMU-based systems use low-cost electronics, are portable, and are therefore more suitable for continuous monitoring in home environments.

The algorithm used to classify sensor data plays a major role in determining the performance of wearable AFD systems. Early approaches mainly relied on threshold-based techniques, where falls were detected when acceleration values crossed a fixed limit. However, these methods often produced high false positives because normal daily activities (ADLs) such as standing up quickly, climbing stairs, or making sudden movements can temporarily exceed these thresholds. To improve reliability, machine learning methods like HMMs and SVMs were later introduced. These methods do not rely strictly on fixed thresholds, but they still require careful feature engineering and domain knowledge to extract useful patterns from the data. More recently, deep learning techniques such as 1D-CNNs have been adopted. These models can automatically learn spatial and temporal patterns directly from sensor data, removing

the need for manually designed features and generally providing better generalization and performance. The SisFall dataset helps address this limitation because it includes data collected from elderly participants, making it more suitable for developing realistic and robust fall detection models.

1.2 Problem Statement

While substantial progress has been achieved in automatic fall detection utilizing approaches such as the hybrid HMM approach and representative datasets (SisFall), there are still key limitations that prevent the development of fully robust and efficient systems. Existing solutions rely heavily on conventional machine learning models (such as HMMs), which are inherently constrained due to their reliance on manual feature engineering. There is no defined, effective methodology for deploying fall detection models on resource-constrained microcontrollers. Prior research has frequently concentrated on different aspects of the problem i.e. the method, the dataset, or the hardware in isolation as a result considerable gap remains in combining these components into a single, cohesive, end-to-end solution that employs a modern deep learning approach.

This project will address these interconnected issues by designing, building, and validating a complete, real-time fall detection system that integrates a modern deep learning architecture (1D-CNN) with an industry-standard deployment framework (TensorFlow Lite for Microcontrollers), resulting in a solution that is more accurate, power-efficient, and aligned with current Edge AI best practices.

1.3 Objective

The main objective of this system is:

- To design and develop a full, real-time wearable system that detects falls more accurately and efficiently on-device than classical methods.

The specific objectives that are going to be met through project are as follow:

1. To use the SisFall dataset to develop 1D-CNN in order to determine the most effective and precise architecture for fall detection.
2. To implement an IoT alert system utilizing the ESP32's Wi-Fi to transmit real-time fall notifications to a caregiver's smartphone via a cloud platform.

1.4 Scope

The scope of this project is to design, develop, and deploy a real-time fall detection system for elderly individuals using a wearable device based on a lightweight 1D Convolutional Neural Network (1D-CNN). The system focuses on accurate and low-latency detection of falls while operating on resource-constrained hardware, specifically the ESP32 microcontroller paired with an MPU-6050 sensor.

The scope of the project comprises the following important aspects:

- Preprocessing IMU sensor data from the SisFall dataset, training a 1D-CNN model.
- Deploying the trained model on the ESP32 for on-device inference, enabling real-time fall detection without relying on cloud processing.
- Sending immediate local and remote notifications via Wi-Fi when a fall is detected.
- Ensuring low power consumption, minimal latency, and a practical wearable form factor suitable for everyday use.

2. LITERATURE REVIEW

Lim et al. (2014) [2] developed a fall detection system that combines a basic threshold method with an HMM based on 3-axis accelerometer data. This hybrid technique was designed to improve fall detection accuracy while keeping hardware complexity to a minimum. A wearable device was created to capture acceleration data, from which certain fall-related metrics could be derived. In order to differentiate real falls from everyday activities, the basic threshold method first filters possible fall events, which are subsequently further examined using two different kinds of HMMs. Experimental results showed that the combined method beat the single threshold approach in terms of detection accuracy, making it ideal for real-time wearable applications focused on senior safety.

Pannurat et al. (2014) [3] conducted a thorough assessment of existing fall detection and monitoring systems, emphasizing their vital significance in protecting the older population. The study divides existing fall detection technologies into two categories: ambient-based systems (such as cameras and floor sensors) and wearable devices (such as gyroscopes and accelerometers). It further divides detection strategies into rule-based methods and machine learning approaches, examining their advantages and disadvantages. The evaluation emphasizes important industry concerns such as assuring accuracy, decreasing false alarms, maintaining user comfort, and integrating technologies into real-world contexts. The study emphasizes the need for dependable, discrete, and affordable ways to enhance senior care, particularly for individuals who live alone.

Sucerquia et al. (2017) [4] presented SisFall, a comprehensive and publicly available dataset for testing fall detection systems with wearable sensors. SisFall, unlike previous datasets, collects data from both young and elderly participants, making it more reflective of real-world settings. The dataset was obtained using a unique wearable device equipped with two accelerometers and a gyroscope, and it included 19 types of ADLs and 15 types of falls. The accuracy of the first tests, which used threshold-based classification and common feature extraction, was up to 96%; however, performance significantly declined when applied to older participants. This emphasizes the need for more robust and elderly-specific fall detection techniques, and SisFall serves as a crucial standard for future wearable healthcare research.

Sucerquia et al. (2018) [5] present a functional fall detection system based on triaxial accelerometers that is suitable for real-world use. The suggested method combines a nonlinear classification feature with a Kalman filter and a periodicity detector in order to reduce false positives while retaining high accuracy. Notably, the system runs well at a modest sampling rate of 25 Hz, making it computationally and memory-lightweight ideal for wearable electronics. The method was first evaluated on the SisFall dataset, where it achieved an accuracy of 99.4%, before moving on to simulations with both young and old subjects. The method was further verified in full-day real-life experiments with elderly people, confirming its resilience, reliability, and suitability for continuous monitoring in everyday settings.

3. FEASIBILITY STUDY

3.1 Technical Feasibility

The proposed fall detection system makes use of an ESP32 microcontroller, an MPU-6050 motion sensor, and a lightweight deep learning model (1D-CNN) implemented using TensorFlow Lite. All of the hardware components required are low-power, inexpensive, and may be obtained from the college's Electronics Lab or local markets. The model training phase takes place on conventional laptops or in cloud environments, whereas the deployment phase is designed for microcontrollers with limited resources. The modular architecture of both software and hardware allows for seamless system integration and development.

3.2 Economic Feasibility

The proposed project is financially feasible for academic implementation, with little costs and a good return on value:

- **Component Costs:** The ESP32, MPU-6050, LiPo battery, and other modules are reasonably priced and are readily available.
- **Development Costs:** Open-source frameworks (TensorFlow, Arduino IDE) are used to reduce software expenses.
- **Operational Costs:** The device consumes less power and functions effectively and automatic fall detection reduces the need for regular supervision or additional people.

4. PROJECT METHODOLOGY

4.1 Block Diagram of the developed System

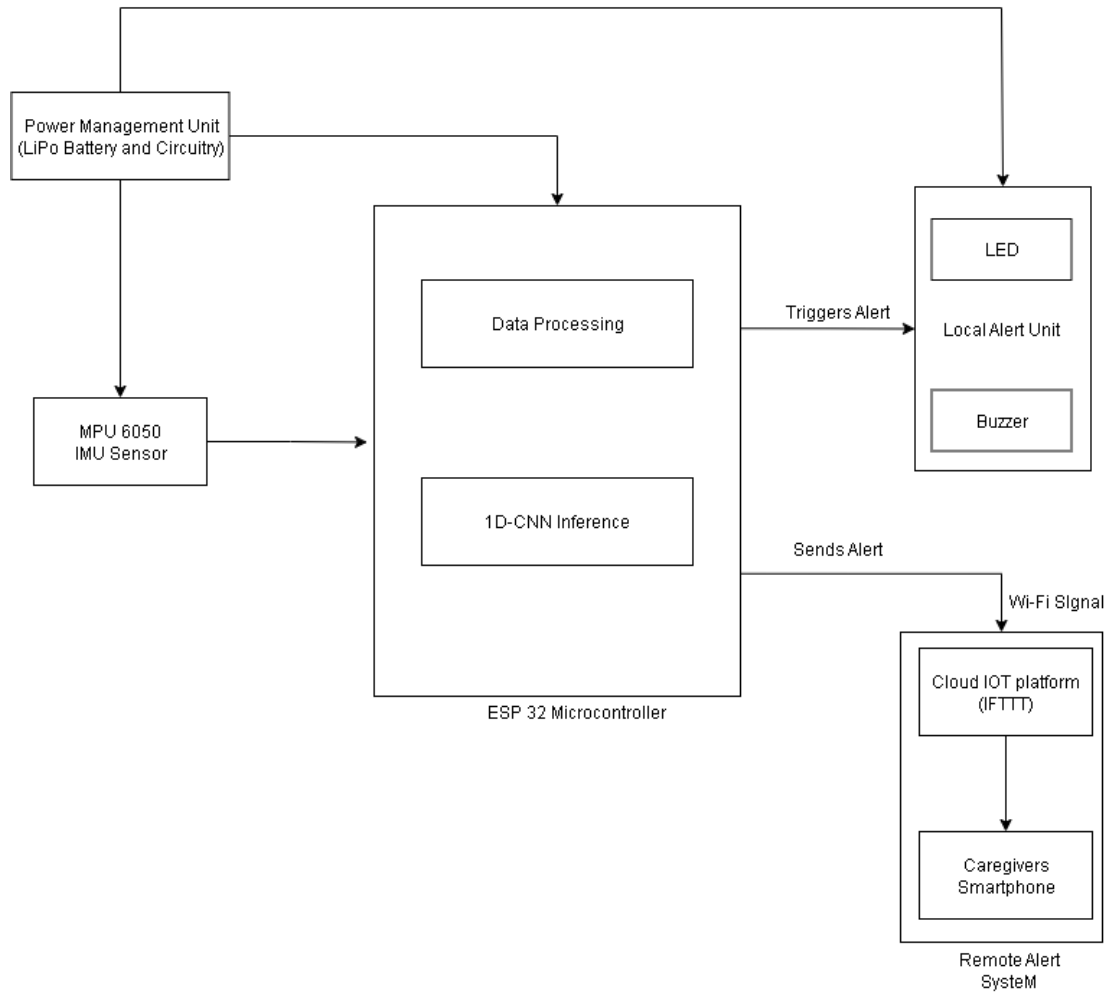


Figure 4.1: Block Diagram

The basic block diagram of fall detection system is shown above which consists of an intelligent wearable device and a remote alert system. The Wearable Device is designed around an ESP32 Microcontroller, which interprets real-time motion data from an MPU-6050 sensor. Onboard software processes data and detects falls accurately using a 1D-CNN inference model. Upon detection, the ESP32 activates a Local Alert Unit (LED and Buzzer) and sends a signal to the Remote Alert System via its Wi-Fi module. This remote system employs a Cloud IoT Platform as a bridge to instantaneously send a push notification to a caregiver's smartphone. An integrated Power Management equipment, which incorporates a rechargeable battery, allows the wearable equipment to be portable and self-sufficient.

4.2 Flowchart of the Developed System

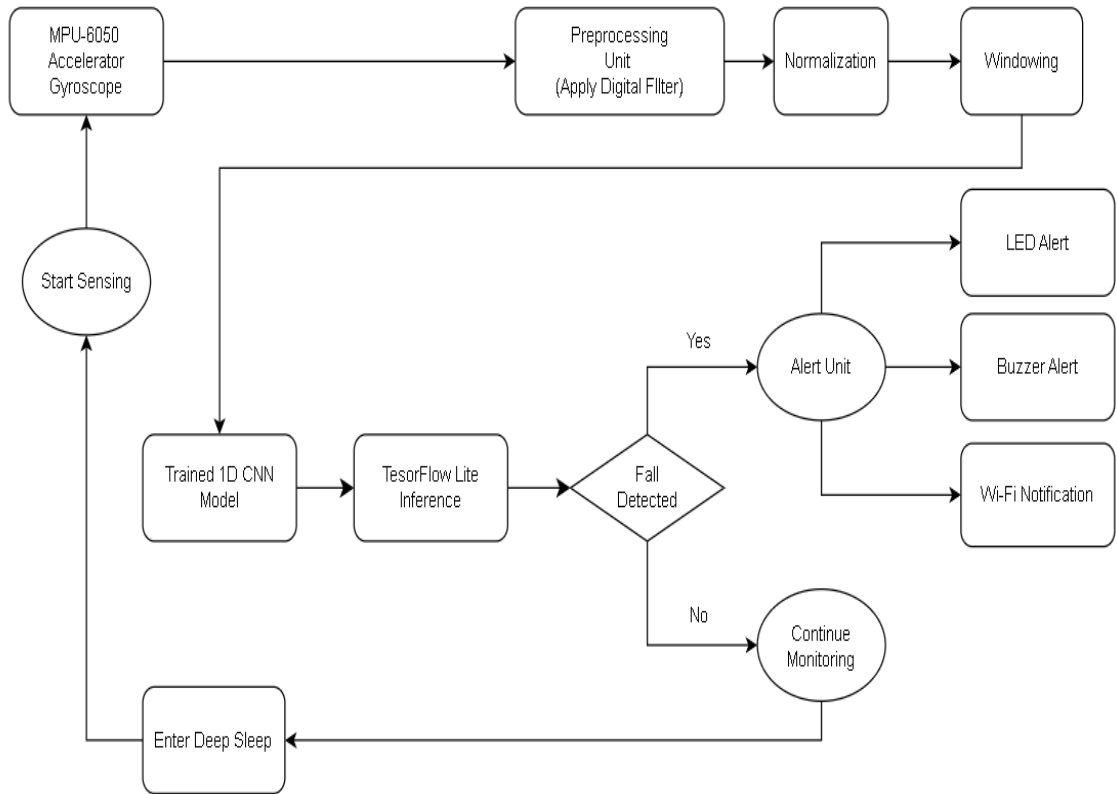


Figure 4.2: Flowchart

The system runs in a continuous, power-efficient loop, starting with the MPU-6050 sensor gathering motion data. This raw data is immediately transferred through an on-device preparation pipeline that includes digital filtering, normalization, and segmentation into time windows. Each full data window is passed into the TensorFlow Lite inference engine, which uses a pre-trained 1D CNN model to classify the activity. When a fall is detected with high confidence, an Alert Unit is activated, triggering local LED and buzzer alarms as well as a remote Wi-Fi message. If no fall is detected, the device enters a brief Deep Sleep mode to preserve battery power before waking up to begin the next sensing cycle, ensuring reliable and ongoing monitoring.

4.3 Model Classification

- **Baseline Simple Threshold Model**

This model serves as the most basic baseline for establishing a performance benchmark. It works on the basis that a fall event is distinguished by a considerable, sudden shift in the body's acceleration.

- **Concept:** A fall is detected when the magnitude of the acceleration vector exceeds a predetermined threshold. This method is computationally simple and very fast.
- **Design and Execution:**
- **Input:** Raw 3-axis accelerometer data (x, y, z) .
- **Feature Calculation:** Determine the Sum Vector Magnitude at each time step using this formula:

$$\text{SVM} = \sqrt{(x^2 + y^2 + z^2)}$$

- **Decision Logic:** The core of the model is a single if statement:

if $(\text{SVM} > \text{THRESHOLD_VALUE})$ then Fall_Detected
- **Establishing the power source Threshold:** The THRESHOLD_VALUE is not determined arbitrarily. It will be empirically determined by plotting a ROC curve against the validation dataset. This enables us to pick a threshold that strikes the best balance between sensitivity (detecting all falls) and specificity (avoiding false alarms).

- **1D CNN Model**

A 1D convolutional layer is a type of neural network layer that performs convolution operations on one-dimensional data. Unlike 2D convolutions that slide a kernel over a two-dimensional input (e.g., images), 1D convolutions slide the kernel along a single spatial dimension.

Input Data for 1D Convolution:

- The input is typically represented as a sequence or a signal. Examples include time-series data, text embeddings, and audio waveforms.
- The shape of the input data is (batch_size, sequence_length, num_features).

Working Principle:

- The layer uses a kernel (filter), a small matrix of learnable weights, to extract features from the sequence.
- The kernel slides over the input sequence, performing a dot product at each position.
- The result of this operation is a transformed feature map, capturing patterns in the data. [6]

Architecture of 1D CNN Model:

The typical structure of a 1D CNN is composed of three important layers: 1D convolutional layers, pooling layers, and fully connected layers [12]. In addition to these three layers, there are two more important parameters, which are the dropout layer and activation function.

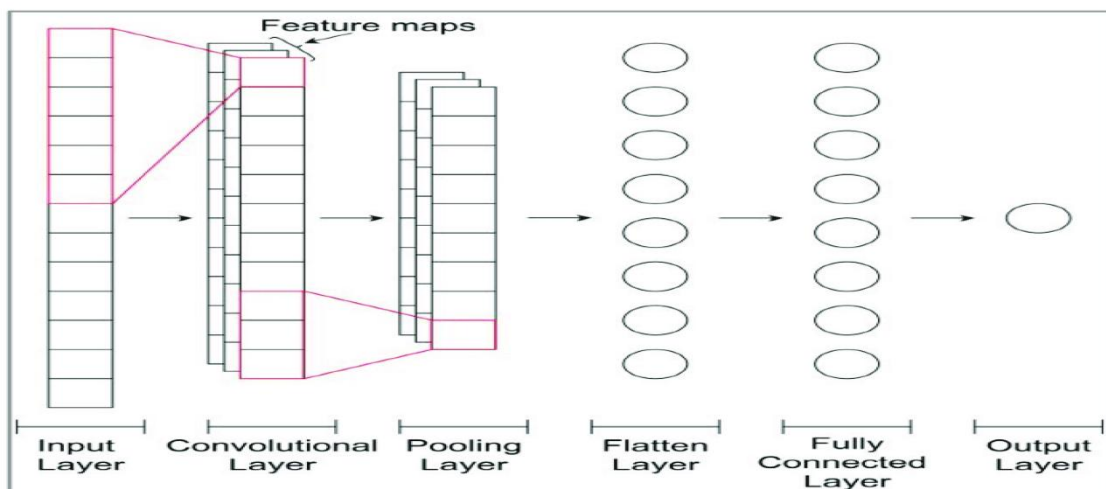


Figure 4.3: Simple 1D convolutional neural network (CNN) architecture

- **One-dimensional convolutional layer [13]:** This is the layer that can be used to detect features in a vector. The raw one-dimensional input (vector) $x[n]$, where $n = 0, 1, \dots, N - 1$ is given as an input to the first layer of the CNN architecture. The layer utilizes the following parameters:
- **Filters or kernels:** The feature maps are the outputs of one filter applied to the previous layer. The filters/kernels produce the feature maps by performing convolutions with the input data. The number and size of the kernels are crucial for adequately capturing the relevant features from the input data. Let $\kappa[n]$ denote the convolution kernel with size Θ , then, the convolution output $\zeta[n]$ can be calculated as

$$\zeta[n] = x[n] * \kappa[n] = \sum_{m=0}^{\Theta-1} \kappa[m] \cdot x[n - m], n = 0, 1, \dots, N - 1 \quad (1)$$

where ‘*’ denotes the convolution operation. In general, the convolved features at the output of the l th layer can be written as

$$\zeta_i^l = \sigma \left(b_i^l + \sum_j \zeta_j^{\{l-1\}} \kappa_{ij}^l \right) \quad (2)$$

where ζ_i^l represents the i th feature in the l th layer, $\zeta_j^{\{l-1\}}$ denotes the j th features in the $(l-1)$ th layer, κ_{ij}^l represents the kernel linked from the i th to the j th features, b_i^l denotes the bias for these features, and σ represents the activation function.

- **Activation function (R) [13]:** One of the most important parameters of the CNN model is the activation function, which is used to learn and approximate any kind of continuous and complex relationship between the variables of the network. There are several activation functions such as the RELU, softmax, and sigmoid functions. In this work, we use the exponential linear unit or its widely known name the ELU, which is an activation function based on the RELU that has an extra alpha constant (α) that defines function smoothness when the inputs are negative. It is a function that tends to converge cost to zero faster and produces more accurate results.

Its formula is with $\alpha > 0$

$$R(\zeta) = \begin{cases} \zeta & \text{if } \zeta > 0 \\ \alpha \cdot (\exp(\zeta) - 1) & \text{if } \zeta \leq 0 \end{cases} \quad (3)$$

- **Strid [14]:** The strid value defines how the kernel moves in the input data. The most common value is 1, meaning that the kernel moves over one column of the input data at each iteration.
- **Pooling Layer [13]:** This type of layer is often placed after the convolutional layer. The aim of this layer is to decrease the size of the convolved features map to reduce computational costs. There are several types of pooling operations (max pooling, average pooling, sum pooling) [14]. In this work, we used 1D max pooling, which consists of running the input with a defined spatial neighborhood or specified pool size and strid, taking the maximum value from the considered region. Its operation can be represented by

$$\zeta_h^l = \max_{\{p \in r_h\}} \zeta_p^{\{l-1\}} \quad (4)$$

where r_h denotes the pooling region with index h .

- **Flatten layer and dropout layer [13]:** The flatten layer transforms the input data into a one-dimensional vector to be fed to the fully connected/dense layer. A dropout parameter is added after the flatten layer; however, when all the features are connected to the flatten layer, it can cause overfitting in the training dataset. To overcome this problem, a dropout layer is utilized, wherein a few neurons are dropped from the neural network during the training process, resulting in the reduced size of the model.
- **Dense fully connected layer [13]:** The flattened output is given as an input to the next layer, i.e., the dense fully connected layer, which produces the output. The activation function is one of its parameters. In this work, we use the ELU function, which is described in (4).

5. SYSTEM DESIGN

5.1 Schematic Diagram:

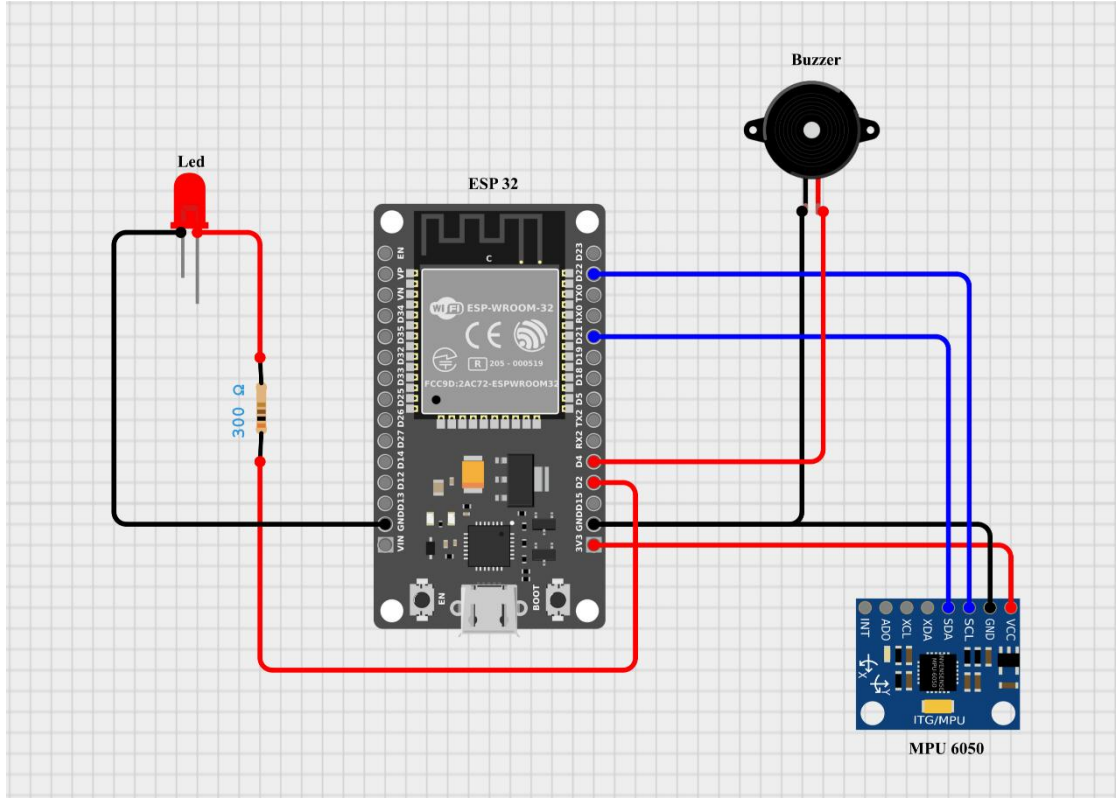


Figure 5.1: Schematic Diagram

The above hardware setup consists of an ESP32 microcontroller acting as the central processing unit, interfaced with an MPU-6050 sensor, a buzzer, and an LED. The MPU-6050 is connected via the I2C protocol, with SDA and SCL linked to GPIO 21 and GPIO 22 of the ESP32, and powered using 3.3V and GND, enabling continuous acquisition of 3-axis accelerometer and gyroscope data. This sensor data is processed by the ESP32, where preprocessing and the deployed 1D CNN model are executed to detect fall events in real time. For alert generation, a buzzer is connected to a GPIO pin (e.g., GPIO 4) to provide an audible alarm, while an LED connected through a 300Ω resistor to another GPIO pin serves as a visual indicator. All components share a common ground, and the ESP32 supplies power to the sensor, forming a compact, real-time embedded fall detection system.

5.2 Model Development:

5.2.1 Dataset Analysis

The SisFall dataset contains 4,510 labeled recordings designed for fall detection and human activity recognition research. Each recording captures nine channels of inertial data sampled at 200 Hz using three onboard sensors the ADXL345 and MMA8451Q accelerometers, and the ITG3200 gyroscope providing three-axis acceleration and three-axis angular velocity readings. Raw values stored in bits are converted to physical units (g and °/s) using each sensor's resolution and range specifications. The dataset spans two broad categories:

- **19 ADL types** including walking, jogging, stair climbing, sitting, lying down, and bending
- **15 fall types** simulating forward, backward, and lateral falls caused by slips, trips, and fainting

Participants Data was collected from 38 volunteers across two age groups:

- 23 young adults (ages 19–30) who performed both ADLs and falls
- 15 elderly individuals (ages 60–75) who primarily performed ADLs, with physically demanding activities excluded on medical advice

This age diversity makes the dataset particularly suitable for building generalizable fall detection models across different populations. Each recording file contains nine columns of raw integer values stored in bits, corresponding to the three onboard sensors. The first six rows of a sample file look like this:

S. N	c0	c1	c2	c3	c4	c5	c6	c7	c8
0	17.0	-179.0	-99.0	-18.0	-504.0	-352.0	76.0	-697.0	-279.0
1	15.0	-174.0	-90.0	-53.0	-568.0	-306.0	48.0	-675.0	-254.0
2	1.0	-176.0	-81.0	-84.0	-613.0	-271.0	-2.0	-668.0	-221.0
3	-10.0	-180.0	-77.0	-104.0	-647.0	-227.0	-34.0	-697.0	-175.0
4	-21.0	-191.0	-63.0	-128.0	-675.0	-191.0	-74.0	-741.0	-133.0
5	-37.0	-225.0	-59.0	-146.0	-700.0	-159.0	-110.0	-840.0	-103.0

Table 5.2.1.1: Raw SisFall Data

Columns c0–c2 are ADXL345 accelerometer axes (X, Y, Z), c3–c5 are ITG3200 gyroscope axes, and c6–c8 are MMA8451Q accelerometer axes all in raw bit form. These values were converted into physical units using each sensor's resolution and range:

- **Acceleration (ADXL345):** multiplied by $(2 \times 16) / 2^{13} \times 9.80665 \rightarrow$ gives m/s^2
- **Gyroscope (ITG3200):** multiplied by $(2 \times 2000) / 2^{16} \rightarrow$ gives deg/s , then converted to rad/s

Only the ADXL345 axes were retained as the chosen accelerometer for this pipeline. After conversion, the nine raw bit columns collapse into six physically meaningful channels per timestep across 19,999 samples:

S. N	ax	ay	az	gx	gy	gz
0	0.651223	-6.85699	-3.79242	-0.01918	-0.53689	-0.37497
1	0.574608	-6.66546	-3.44765	-0.05646	-0.60507	-0.32597
2	0.038307	-6.74207	-3.10289	-0.08948	-0.65301	-0.28869
3	-0.38307	-6.8953	-2.94966	-0.11079	-0.68923	-0.24182
4	-0.80445	-7.31668	-2.41336	-0.13635	-0.71905	-0.20347
5	-1.41737	-8.61913	-2.26013	-0.15553	-0.74569	-0.16938

Table 5.2.1.2: Converted SisFall Data

5.2.2 Data Acquisition and Preprocessing

- **Sensor Sampling and Physical Unit Conversion**

The SisFall data set was loaded and checked to provide consistency and correctness of data. Sensor data was acquired at precisely 200 Hz, enforced by a `millis()`-gated loop on Core 0. At each 5 ms tick, 14 bytes are burst-read from MPU-6050 registers 0x3B–0x48. After subtracting per-axis calibration offsets, raw LSB values are converted to physical units:

- Accelerometer: $a \text{ [m/s}^2] = (\text{raw_LSB} / 2048) \times 9.80665$
- Gyroscope: $\omega \text{ [rad/s]} = (\text{raw_LSB} / 16.4) \times (\pi / 180)$

These conversions match the physical-unit representation applied during training on SisFall, ensuring a consistent input distribution between training and inference. This earned 74,409 labeled windows. The data had been divided into 80 to 20 ratio training and validation datasets.

- **Sliding Window Segmentation**

Samples were accumulated in a circular `sensorBuffer` of size 400×6 . At 200 Hz, 400 samples correspond to a 2-second observation window; the canonical duration in the SisFall literature for capturing the pre-impact, impact, and post-impact phases of a fall. A 50% overlap strategy is applied: when the buffer fills, the second half (samples 200–399) is retained as the first half of the next window using a single `memcpy` (approximately $10\times$ faster than element-wise copying), and the buffer write index resets to 200. This yields one inference per second while maintaining continuous temporal context

- **Z-Score Normalization and INT8 Quantization**

Fall events are less frequent than daily activities, so the distribution of classes was studied. Before being passed to the model, each channel value was Z-score normalized using training-set statistics computed on the SisFall training split. Values beyond $\pm 4\sigma$ were soft-clamped. The normalized float was then linearly quantized to INT8 using the model's input scale and zero point derived from post-training quantization.

Channel	Physical Quantity	Unit	Mean	Std Dev
ax	Accelerometer X	m/s ²	−0.0461	4.0721
ay	Accelerometer Y	m/s ²	−6.8318	5.8187
az	Accelerometer Z	m/s ²	−1.1165	4.8141
gx	Gyroscope X	rad/s	−0.01996	0.44048
gy	Gyroscope Y	rad/s	−0.74466	0.62880
gz	Gyroscope Z	rad/s	−0.06945	0.54259

Table 5.2.2: Per-channel Z-score normalization statistics

- SisFall recordings processed: 4,505
- Window generation: 74,409 windows
 - Class 0 (Non-fall): 49359 windows
 - Class 1 (Fall): 25050 windows
- Window size: 2 seconds (400 samples)
- Window overlap: 50%
- Data split:
 - Training: 59,527 (80%)
 - Validation: 14,882 (20%)
- Class distribution:
 - Falls: 33.7%
 - Non-Falls: 66.3%
- Normalization: Z-score standardization

5.2.3 Model Trained using 1D-CNN

An architecture of the lightweight 1D-CNN was applied to learn temporal patterns from time-series sensor data. The 1D CNN architecture was designed for deployment on microcontroller-class hardware, targeting a parameter count below 50,000 and a TFLite binary below 60 KB. The architecture; confirmed via the Kaggle training notebook; comprises three convolutional blocks with progressively increasing filter depth and decreasing kernel size, followed by Global Average Pooling, a Dropout layer, and a sigmoid output. The model was implemented in TensorFlow 2.19.0 / Keras and trained on a dual-GPU Kaggle environment. Techniques that were applied to minimize overfitting and the cost of computation include pooling, normalization, and regularization.

S. N	Layer / Block	Output Shape	Params	Notes
0	Input Layer	(400, 6)	0	400 timesteps $\times$ 6 IMU channels
1	Conv1D — Block 1	(400, 32)	1,376	32 filters, kernel=7, ReLU, same padding
2	BatchNormalization	(400, 32)	128	Trainable: 64, non-trainable: 64
3	MaxPooling1D	(200, 32)	0	pool_size=2, halves temporal dimension
4	Conv1D — Block 2	(200, 64)	12,352	64 filters, kernel=5, ReLU, same padding
5	BatchNormalization	(200, 64)	256	Trainable: 128, non-trainable: 128
6	MaxPooling1D	(100, 64)	0	pool_size=2
7	Conv1D — Block 3	(100, 128)	24,704	128 filters, kernel=3, ReLU, same padding
8	BatchNormalization	(100, 128)	512	Trainable: 256, non-trainable: 256
9	GlobalAveragePooling1D	(128,)	0	Collapses temporal dimension; no parameters
10	Dropout (rate=0.4)	(128,)	0	Regularization before output layer
11	Dense — Output	(1,)	129	Sigmoid activation — fall probability $\in [0, 1]$
TOTAL	—	—	37,409	Trainable: 36,961 non-trainable: 448

Table 5.2.3: 1D CNN layer-by-layer architecture

The three convolutional blocks use decreasing kernel sizes ($7 \rightarrow 5 \rightarrow 3$), enabling hierarchical temporal feature extraction: large kernels in the first block capture broad motion trends (e.g., orientation change during a fall onset), while smaller kernels in deeper blocks capture fine-grained local dynamics (e.g., impact transients). The filter depth increases from 32 to 128, following the standard pyramidal design. Global Average Pooling replaces a Flatten operation, eliminating additional dense parameters and providing translation invariance. The single sigmoid output neuron produces a scalar fall probability in $[0, 1]$, enabling a tunable confidence threshold at inference time.

5.2.4 Model Training on Kaggle GPU Environment

The model was trained in TensorFlow 2.19.0 on the Kaggle kernel environment equipped with two physical GPUs. Training was configured with the following hyperparameter settings:

Hyperparameter	Value
Optimizer	Adam
Initial learning rate	1×10^{-3}
Batch size	64
Maximum epochs	70
Class weighting	Balanced (sklearn. utils.class_weight)
LR scheduler (ReduceLROnPlateau)	factor=0.5, patience=4, monitor=val_loss, min_lr= 1×10^{-6}
Early stopping	patience=10, monitor=val_loss, restore_best_weights=True
Model checkpoint	Save best only (monitor=val_loss)
Train/Validation split	80% / 20% stratified (sklearn.model_selection.train_test_split)
Loss function	Binary cross-entropy (class-weighted)

Table 5.2.4.1: Model training hyperparameters.

Class-weighted binary cross-entropy was employed to mitigate the class imbalance (fall: 33.7%, non-fall: 66.3%), upweighting the fall class by approximately 2:1. The ReduceLROnPlateau callback halved the learning rate whenever validation loss failed to improve for 4 consecutive epochs; the learning rate decayed through the schedule $1 \times 10^{-3} \rightarrow 5 \times 10^{-4} \rightarrow 2.5 \times 10^{-4} \rightarrow 1.25 \times 10^{-4} \rightarrow 6.25 \times 10^{-5} \rightarrow 3.125 \times 10^{-5}$ across 70 epochs. Early stopping monitored validation loss with a patience of 10 epochs and restored the best-observed weights; the best checkpoint corresponded to epoch 70, confirming that the model continued to improve throughout training without overfitting.

Epoch	LR	Train Acc	Train AUC	Train Loss	Val Acc	Val AUC	Val Loss
1	1.00×10^{-3}	0.7703	0.8095	0.5356	0.8089	0.8639	0.4787
5	1.00×10^{-3}	0.8340	0.8947	0.4057	0.8434	0.9146	0.3693
10	1.00×10^{-3}	0.8536	0.9297	0.3356	0.8688	0.9401	0.2942
16	5.00×10^{-4} ↓	0.8805	0.9540	0.2726	0.8912	0.9609	0.2450
25	5.00×10^{-4}	0.8936	0.9624	0.2484	0.8957	0.9699	0.2361
35	2.50×10^{-4} ↓	0.9083	0.9718	0.2147	0.9169	0.9774	0.1907
40	1.25×10^{-4} ↓	0.9142	0.9751	0.2011	0.9216	0.9799	0.1829
56	1.25×10^{-4}	0.9176	0.9776	0.1913	0.9245	0.9804	0.1760
58	6.25×10^{-5} ↓	0.9223	0.9792	0.1841	0.9255	0.9820	0.1731
65	3.13×10^{-5} ↓	0.9209	0.9799	0.1815	0.9269	0.9817	0.1723
70	3.13×10^{-5}	0.9210	0.9786	0.1863	0.9285	0.9822	0.1697

Table 5.2.4.2: Selected epoch training log

The above table presents a selection of representative epoch checkpoints extracted from the Kaggle training log, illustrating the convergence trajectory of both training and validation metrics.

5.2.5 Quantization and Embedded Deployment

The trained Keras model (sisfall_1dcnn_model.h5) was converted to TensorFlow Lite Micro format using INT8 post-training quantization. A representative calibration dataset of 100 validation samples was provided to the TFLite converter to determine per-tensor scale factors and zero points for all activations and weights. The quantized binary was produced with TFLiteConverter, confirming INT8 input and output types, and serialized to sisfall_model.tflite. The file was then converted to a C byte array using the xxd command-line tool and embedded as fall_detection_model.h in the ESP32 firmware. The quantization process produced the following exact parameters, verified by loading the TFLite model through tf.lite.Interpreter:

Parameter	Value	Usage
Input shape	[1, 400, 6]	1 sample $\times$ 400 timesteps $\times$ 6 channels
Input data type	INT8	Signed 8-bit integer
Input scale (INPUT_SCALE)	0.05181732773780823	Float $\rightarrow$ INT8 quantization divisor
Input zero point (INPUT_ZERO_POINT)	37	INT8 offset for input tensor
Output shape	[1, 1]	Scalar fall probability
Output data type	INT8	Signed 8-bit integer
Output scale (OUTPUT_SCALE)	0.00390625 (1 / 256)	INT8 $\rightarrow$ float dequantization factor
Output zero point (OUTPUT_ZERO_POINT)	-128	INT8 offset for output tensor
Model file size	53.26 KB (54,536 bytes)	TFLite binary after INT8 quantization
TFLite accuracy (100 samples)	94.00%	On-device inference validation
Tensor arena (ESP32 SRAM)	40,000 bytes	TENSOR_ARENA_SIZE in config.h

Table 5.2.5: TFLite INT8 quantization parameters

The TFLite model achieved 94.00% accuracy on a 100-sample validation subset tested through the TFLite interpreter, confirming that INT8 quantization introduces negligible accuracy degradation relative to the full-precision Keras model. On the ESP32, AllocateTensors() and Invoke() execute entirely within the 40 KB arena, with Core 1 completing each inference in under 50 ms well within the 1,000 ms window generation period, ensuring the inference pipeline never becomes a throughput bottleneck.

5.2.6 Evaluation Metrics for Fall detection

Evaluation for output of Detection is done using F1 Score analysis. It is determined from confusion matrix as:

		Predicted	
		Negative	Positive
Actual	Negative	True Negative	False Positive
	Positive	False Negative	True Positive

Figure 5.2.6: Confusion Matrix

- **TN (True Negative):** Correctly predicted No Fall
- **FP (False Positive):** Wrongly predicted Fall (False alarm)
- **FN (False Negative):** Missed actual Fall (Dangerous)
- **TP (True Positive):** Correctly detected Fall

5.2.6.1 Precision:

Precision is a good measure to determine, when the costs of False Positive is high. Precision talks about how precise/accurate a model is out of those predicted positive, how many of them are actual positive. It is mainly concerned about, when it predicts fall, how often is it right?

$$Precision = \frac{TP}{TP + FP} = \frac{TP}{Total\ Predicted\ Positive}$$

5.2.6.2 Recall:

Recall actually calculates how many of the Actual Positives our model capture through labeling it as Positive (True Positive). It is mainly concerned about, of all actual falls, how many did we catch?

$$Recall = \frac{TP}{TP + FN} = \frac{TP}{Total\ Actual\ Positive}$$

5.2.6.3 F1 Score:

F1 is a function of Precision and Recall. F1 Score is needed when you want to seek a balance between Precision and Recall. It is a balanced measure of Precision and Recall.

$$F1\ Score = \frac{2 * Precision * Recall}{Precision + Recall}$$

6. HARDWARE AND SOFTWARE REQUIREMENT

6.1 Hardware Requirement

- **ESP32 microcontroller (dual-core, Wi-Fi enabled):** It acts as the main controller that processes sensor data, runs the 1D CNN model, and handles alerts and communication.
- **MPU-6050 IMU sensor (accelerometer + gyroscope):** It collects real-time motion data (acceleration and angular velocity) used to detect fall events.
- **Buzzer (for audible alert):** It produces a sound when a fall is detected to notify nearby people.
- **LED with 300Ω resistor (for visual indication):** It provides a visual alert; the resistor limits current to protect the LED.
- **Breadboard (for circuit assembly):** It is used to assemble and organize the circuit components during prototyping.
- **Power supply (USB cable or Li-Po battery):** It provides power to the system for operation, supporting both testing and portable use.

6.2 Software Requirements

Software needed on the PC for model development:

- **Operating System:** Windows 10/11, macOS, or a Linux distribution (e.g., Ubuntu).
- **Programming Language:** Python (version 3.8 or newer).
- **Core Python Libraries:** TensorFlow & Keras, Scikit-learn, Pandas, NumPy, Matplotlib & Seaborn
- **Integrated Development Environment (IDE):** Visual Studio Code, PyCharm, or Jupyter Notebooks.

Software needed to program the ESP32 microcontroller:

- **Programming Language:** C++.
- **Development Toolchain:** Arduino IDE and VS code
- **Core C++ Libraries:** TensorFlow Lite for Microcontrollers, Adafruit MPU-6050 Library (or equivalent), WiFi.h & HTTPClient.h.

7. RESULT AND DISCUSSION

7.1 Dataset Processing Summary

The SisFall dataset was processed by the pipeline, reading 4,506 files from the dataset root and successfully converting 4,505 recordings to labelled sensor windows (one Readme.txt was skipped due to non-numeric content).

Parameter	Value
SisFall recordings found	4,506 files (4,505 data + 1 readme)
Total labelled windows generated	74,409
Window size	400 samples $\times$ 6 channels (2 s at 200 Hz)
Window overlap (step fraction)	50% (STEP_FRAC = 0.5, step = 200 samples)
Training set (80% stratified)	59,527 windows (931 batches $\times$ batch size 64)
Validation set (20% stratified)	14,882 windows
Non-fall windows (Class 0)	49,359 (66.3% of total)
Fall windows (Class 1)	25,050 (33.7% of total)
Normalization method	Z-score per channel (training-set mean & std)
Accelerometer used	ADXL345 (accel choice = 1)
Physical conversion — ADXL345	0.003906 g/bit $\rightarrow \times 9.80665$ m/s ²
Physical conversion — ITG3200 gyro	0.061035 deg/s per bit $\rightarrow \times (\pi/180)$ rad/s

Table 7.1: SisFall dataset processing summary.

7.2 Model Accuracy and Loss

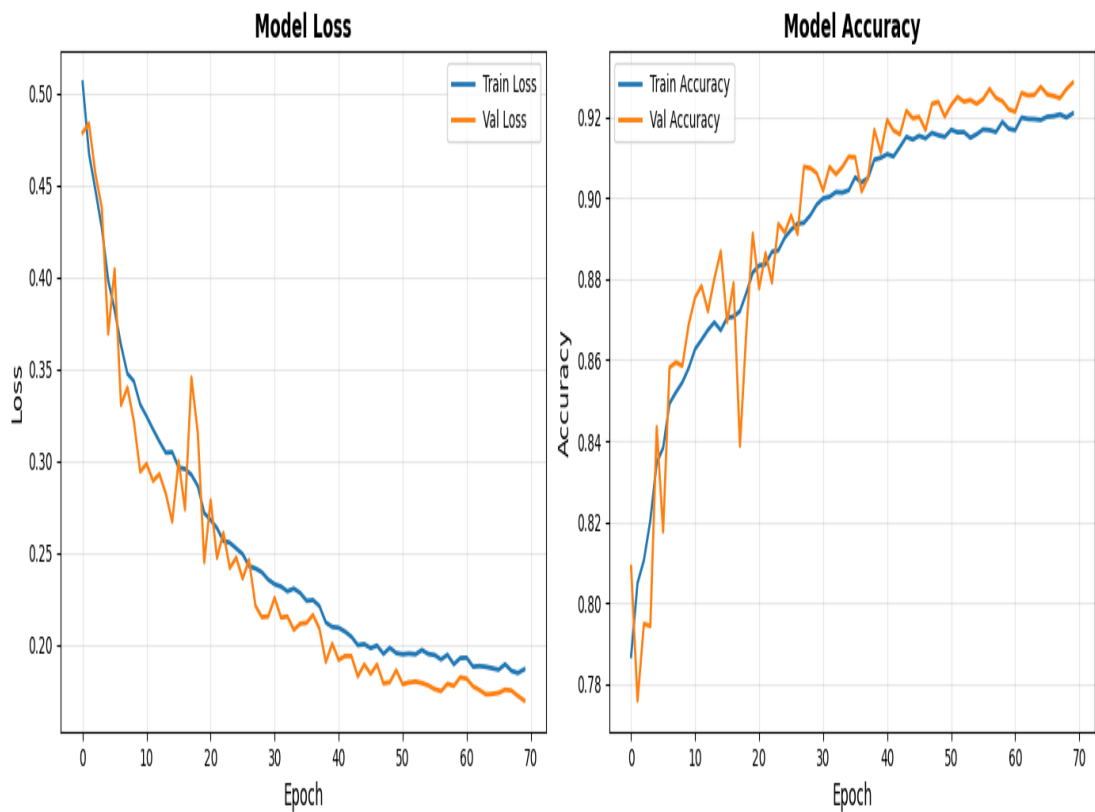


Figure 7.2: Accuracy and Loss Curve of 1D CNN Model

The training and validation curves indicate that the model learned effectively over 70 epochs. Both training and validation loss steadily decreased, converging around 0.19 and 0.17, respectively, while training and validation accuracy increased to approximately 92% and 93%. Early fluctuations in the validation metrics were minimal and stabilized over time, suggesting the model generalizes well to unseen data without overfitting.

7.3 Confusion Matrix and Classification Report

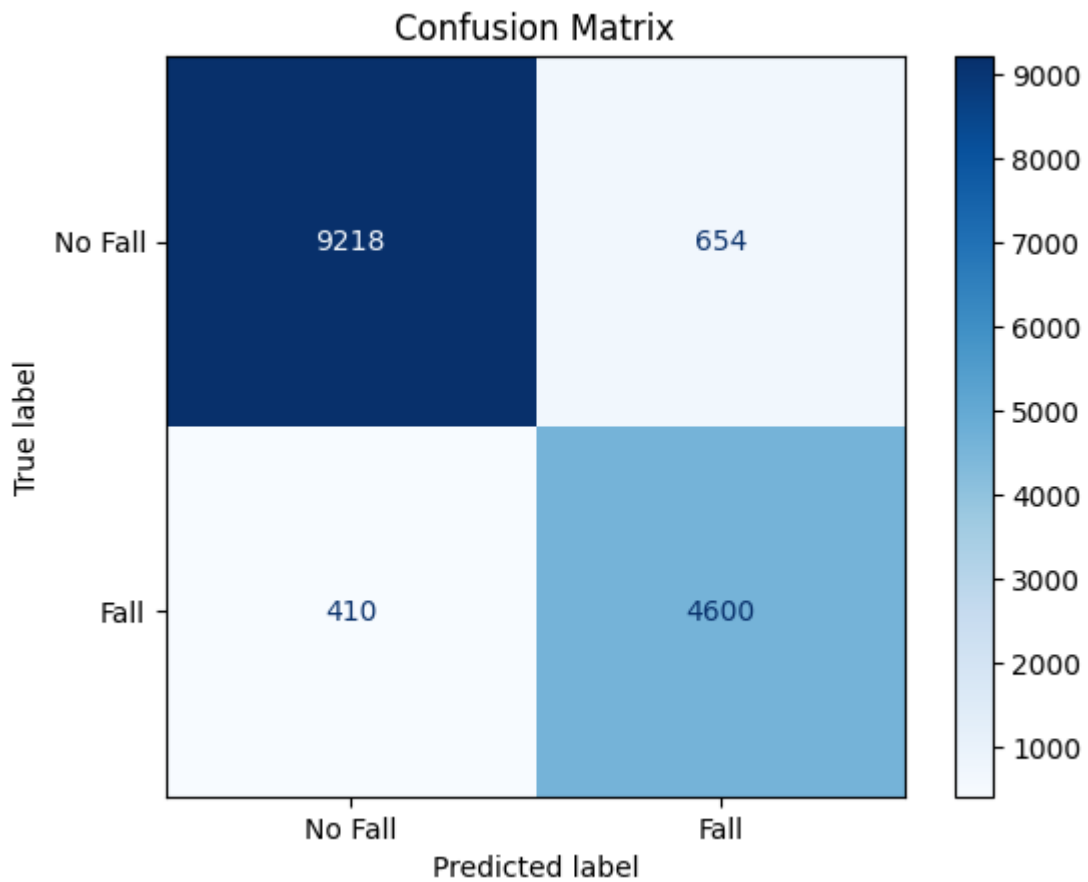


Figure 7.3: Confusion Matrix of 1D CNN Model

Analysis of the confusion matrix reveals strong performance on both classes. The True Positive count of 4,600 confirms that the vast majority of fall events are correctly identified. The False Negative count of 410 corresponds to an 8.2% miss rate, equivalent to the 92% fall recall figure. In the context of elderly fall detection, False Negatives are the most clinically dangerous error type, and a 92% recall rate exceeds the original project target of $\geq 90\%$. The False Positive rate of 6.6% (654 events per 9,872 non-fall windows) is well within tolerable bounds for practical deployment.

- **True Negatives (Correct Non-Falls):** 9,218
- **False Positives (False Alarms):** 654 (6.6%)
- **False Negatives (Missed Falls):** 410 (8.2%)
- **True Positives (Correct Falls):** 4,600

Class	Precision	Recall	F1-Score	Support
Non-Fall (Class 0)	0.96	0.93	0.95	9,872
Fall (Class 1)	0.88	0.92	0.90	5,010
Accuracy (overall)	—	—	0.93	14,882
Macro Average	0.92	0.93	0.92	14,882
Weighted Average	0.93	0.93	0.93	14,882

Table 7.3: Full classification report on the 14,882-window validation set

The classification model demonstrates strong overall performance, achieving an accuracy of 93% on the dataset. For the Non-Fall class, the model shows excellent performance with a precision of 0.96, recall of 0.93, and F1-score of 0.95, indicating that non-fall events are identified both accurately and consistently. For the Fall class, the model achieves a precision of 0.88 and a higher recall of 0.92, resulting in an F1-score of 0.90, which suggests that most fall events are successfully detected, though with slightly more false positives compared to non-fall predictions. The macro and weighted averages, both around 0.92–0.93, further confirm that the model maintains a balanced and reliable performance across both classes, making it effective for fall detection tasks where identifying actual fall events is particularly important.

7.4 Prototype Implementation

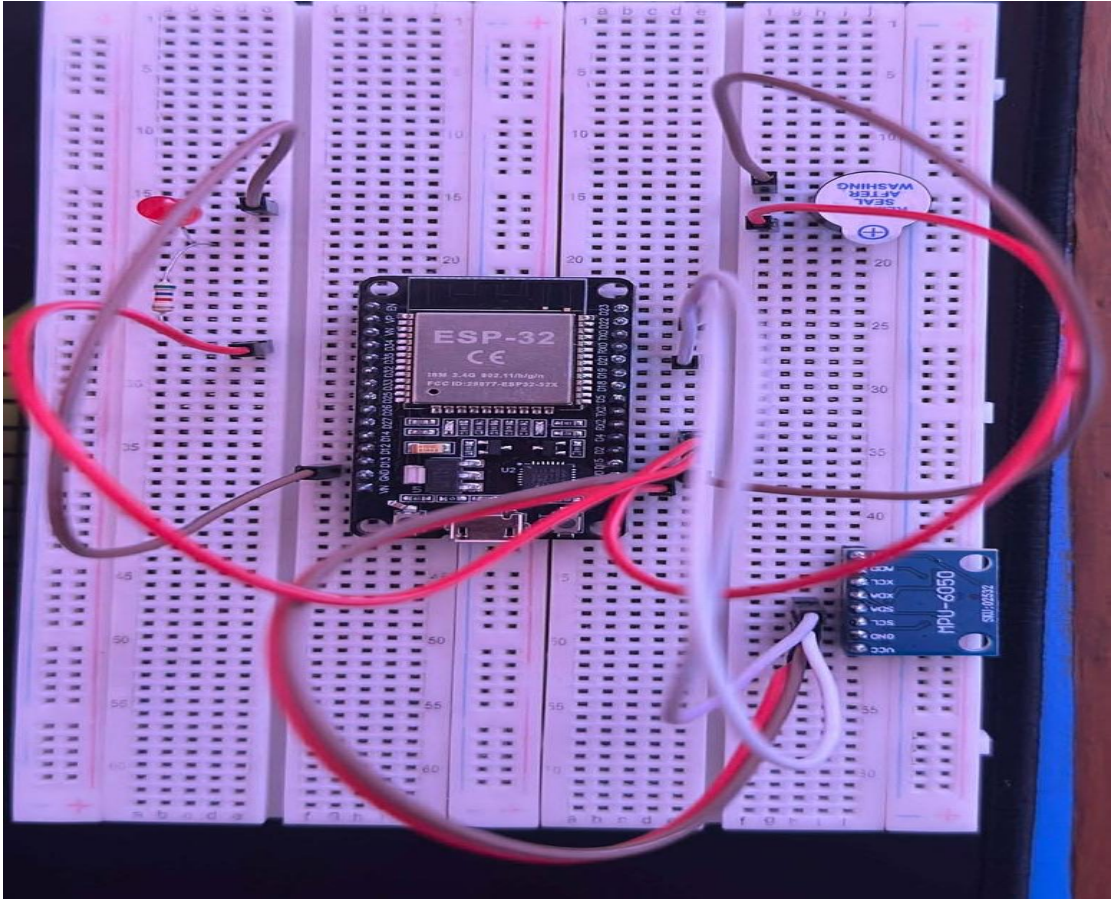


Figure 7.4: Prototype Implementation

The trained and quantized fall detection model was deployed on a physical embedded system to validate its real-world feasibility. The prototype, shown above, was assembled on a breadboard and consists of three primary components working together as a complete edge-inference pipeline.

ESP32 Microcontroller sits at the center of the circuit and serves as the main processing unit. The quantized TensorFlow Lite model (53 KB, INT8) was flashed directly onto the ESP32's flash memory. Its dual-core architecture and sufficient RAM made it capable of running the 1D-CNN inference locally without any cloud dependency, which is critical for a real-time fall detection application.

MPU-6050 IMU Sensor (visible on the lower right of the breadboard) is a combined 3-axis accelerometer and 3-axis gyroscope module, communicating with the ESP32 over I2C. It continuously streams 6-channel inertial data directly mirroring the a_x , a_y , a_z , g_x , g_y , g_z format that the model was trained on. Incoming raw readings are normalized on-device using the channel means and standard deviations computed during training before being fed into the model.

Buzzer and LED (top right and left of the breadboard respectively) serve as the alert output mechanism. When the model's output probability crosses the 0.5 threshold classifying a window as a fall then ESP32 triggers both the buzzer and the LED simultaneously, providing an immediate audible and visual alarm.

The system operates on a sliding window basis, collecting 400 samples (2 seconds at 200 Hz) with 50% overlap, running inference on each window, and outputting a binary fall or non-fall decision in near real-time. The successful deployment of a 93 % accurate model onto a low-cost, compact hardware setup demonstrates that deep learning-based fall detection is practically achievable at the edge without relying on external servers or high-power computing hardware.

7.5 Alert and Notification

The alert hardware and notification system together demonstrated reliable performance in providing timely responses to detected fall events. The passive buzzer and LED were consistently activated, delivering clear audible and visual alerts immediately after a confirmed fall. In addition, the notification system successfully transmitted alerts to the intended recipients, ensuring remote awareness alongside local indication.

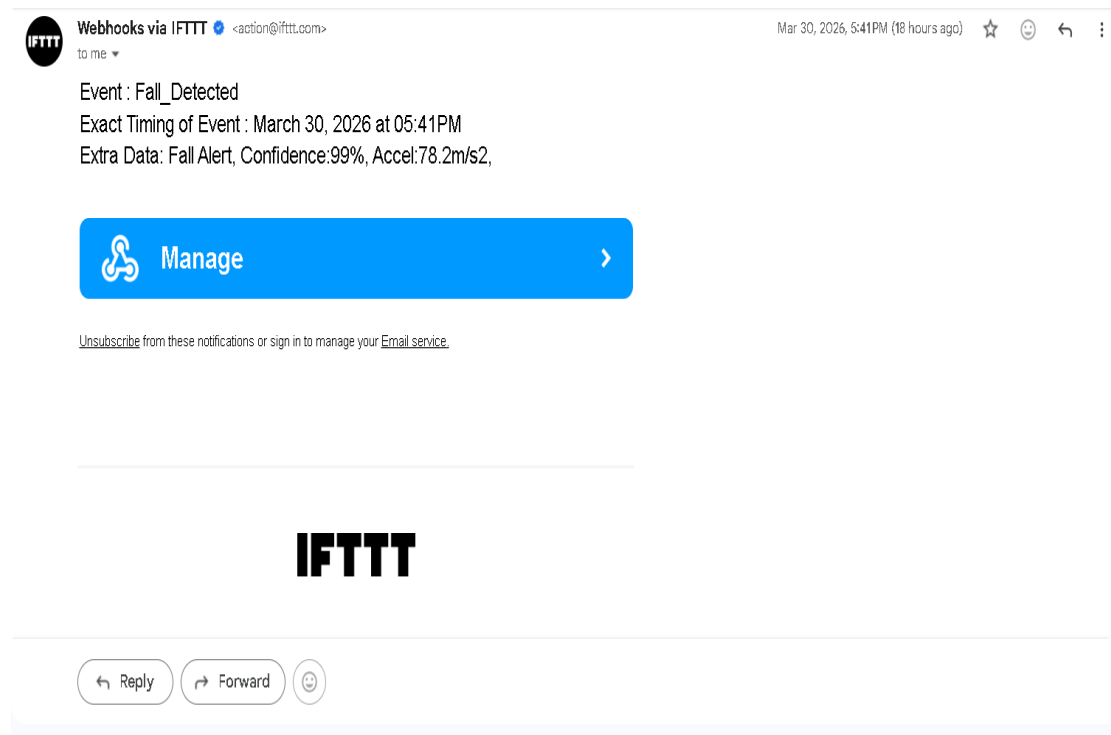


Figure 7.5: Email Alert

8. CONCLUSION

This project successfully designed and deployed real-time elderly fall detection system using a lightweight 1D Convolutional Neural Network on an ESP32 microcontroller. Trained on the SisFall dataset, the model achieved 93% validation accuracy, 92% fall recall, and an F1-score of 0.90, outperforming baseline threshold-based and HMM methods. The integrated dual-gate fall confirmation and debounce filtering ensured reliable local and remote alerts. The study demonstrates that deep learning models can be effectively deployed on resource-constrained hardware to provide an accurate, low-latency, and cost-effective solution for elderly safety.

9. LIMITATION AND FUTURE WORK

A key limitation of the fall detection system was its reliance on the SisFall dataset, which consists of simulated falls performed in controlled laboratory conditions rather than genuine accidental falls in real-world settings. This restricts the model's ability to generalize across the natural variability of how different people particularly elderly individuals with mobility impairments actually fall in everyday environments. Further testing and validation using more diverse, real-world fall data across different body positions and wearing locations is needed to confirm the system's reliability for practical deployment.

Future improvements may involve collecting on-device sensor data to retrain or fine-tune the model specifically for the MPU-6050, improving consistency between training and deployment hardware. Integrating the system with IoT platforms such as MQTT or cloud-based dashboards could further enhance its usefulness by enabling automated emergency alerts to caregivers or medical personnel whenever a fall is detected. Exploring more advanced model architectures such as LSTM or Transformer-based networks may also improve detection accuracy, particularly for edge cases like near-falls and stumbles that closely resemble genuine fall events.

10. REFERENCES

- [1] World Health Organization, "Falls," WHO Fact Sheet, Apr. 2021. [Online].
- [2] D. Lim, C. Park, N. H. Kim, S.-H. Kim, and Y. S. Yu, "Fall-Detection Algorithm Using 3-Axis Acceleration: Combination with Simple Threshold and Hidden Markov Model," *Journal of Applied Mathematics*, vol. 2014, no. 2014, Art. ID 896030, 8 pp., Sep. 2014.
- [3] N. Pannurat, S. Thiemjarus, and E. Nantajeewarawat, "Automatic Fall Monitoring: A Review," *Sensors*, vol. 14, no. 7, pp. 12900–12936, Jul. 2014.
- [4] A. Sucerquia, J. D. López, and J. F. Vargas-Bonilla, "SisFall: A Fall and Movement Dataset," *Sensors*, vol. 17, no. 1, pp. 198, Jan. 2017.
- [5] A. Sucerquia, J. D. López, and J. F. Vargas-Bonilla, "Real-Life/Real-Time Elderly Fall Detection with a Triaxial Accelerometer," *Sensors*, vol. 18, no. 4, pp. 1101, Apr. 2018.
- [6] A. Jain, "Understanding the 1D Convolutional Layer in Deep Learning," *Medium*, Feb. 20, 2024.
- [7] Espressif Systems, "ESP32 Technical Reference Manual," Rev. 5.1, 2023. [Online]. Available: <https://www.espressif.com>
- [8] InvenSense, "MPU-6000 and MPU-6050 Product Specification, Rev. 3.4," TDK Corporation, 2013.
- [9] TensorFlow Team, "TensorFlow Lite for Microcontrollers," Google, 2023. [Online]. Available: <https://www.tensorflow.org/lite/microcontrollers>
- [10] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, pp. 436–444, May 2015.
- [11] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *Proc. ICML*, 2015, pp. 448–456.
- [12] E. Chaerun Nisa and Y. D. Kuan, "Comparative Assessment to Predict and Forecast Water-Cooled Chiller Power Consumption Using Machine Learning and Deep Learning Algorithms," *Sustainability*, vol. 13, p. 744, 2021.

- [13] M. Saini, U. Satija, and M. D. Upadhayay, “Light-Weight 1-D Convolutional Neural Network Architecture for Mental Task Identification and Classification Based on Single-Channel EEG,” *arXiv preprint arXiv:2012.06782*, 2020.
- [14] J. Rala Cordeiro, A. Raimundo, O. Postolache, and P. Sebastião, “Neural Architecture Search for 1D CNNs—Different Approaches: Tests and Measurements,” *Sensors*, vol. 21, p. 7990, 2021.
- [15] S. Guessoum, S. Belda, J. M. Ferrandiz, S. Modiri, S. Raut, S. Dhar, R. Heinkelmann, and H. Schuh, “The Short-Term Prediction of Length of Day Using 1D Convolutional Neural Networks (1D CNN),” *Sensors*, vol. 22, no. 23, p. 9517, 2022.

11.APPENDICES

Outcome of the project

- Initialization

```
13:26:48.502 -> =====
13:26:48.502 -> ELDERLY FALL DETECTION SYSTEM - 1D-CNN
13:26:48.502 -> Kathford College | Bishal, Nishant, Sabin
13:26:48.502 -> =====
13:26:48.502 ->
13:26:48.502 -> 1. Initialising MPU-6050... OK
13:26:48.597 -> Range : ±16 g accel / ±2000 °/s gyro (SisFall-matched)
13:26:48.597 -> DLPF : 44 Hz bandwidth (DLPF_CFG = 3)
13:26:48.597 -> I2C : 400 kHz fast-mode
13:26:48.597 ->
13:26:48.597 -> 2. Calibrating MPU-6050...
13:26:48.597 -> Keep device STILL on a flat surface!
13:26:50.629 -> Collecting 500 samples ..... Done!
13:26:51.588 -> Calibration offsets (LSB):
13:26:51.620 -> Accel: X=-77.5 Y=12.3 Z=-4200.1
13:26:51.620 -> Gyro : X=73.6 Y=36.8 Z=17.7
13:26:51.620 ->
13:26:51.620 -> 3. Loading TensorFlow Lite model...
13:26:51.620 -> Model loaded (54536 bytes)
13:26:51.620 -> Tensors allocated OK
13:26:51.620 -> Input shape : [1, 400, 6] type: INT8
13:26:51.620 -> Output tensor dims: 2 → [1, 1]
13:26:51.620 -> Output type: single SIGMOID (index 0 = fall probability)
13:26:51.620 -> Arena used : 30409 / 40000 bytes (76.0%)
13:26:51.620 ->
13:26:51.620 -> 4. Connecting to WiFi...
13:26:51.666 -> SSID: nishant321_2.4
13:26:52.266 -> ...
13:26:53.255 -> Connected! IP: 192.168.1.122 RSSI: -60 dBm
13:26:53.255 ->
13:26:53.255 -> =====
13:26:53.288 -> SYSTEM READY
```

- When no fall is detected

```
13:27:20.883 -> [Inference #4 Core 1]
13:27:20.883 -> CNN confidence : 1.2% threshold 60% fail
13:27:20.883 -> Peak accel : 121.19 m/s² min 11.0 m/s² PASS
13:27:20.883 -> Decision : normal
13:27:20.883 -> History : [. . .] 0 / 2
13:27:20.883 -> Debounce : monitoring
13:27:20.883 -> Inference time : 6057.59 ms
13:27:26.991 -> Raw output : fall=28.9% (sigmoid)
13:27:26.991 ->
13:27:26.991 -> [Inference #5 Core 1]
13:27:26.991 -> CNN confidence : 28.9% threshold 60% fail
13:27:26.991 -> Peak accel : 9.90 m/s² min 11.0 m/s² fail
13:27:26.991 -> Decision : normal
13:27:26.991 -> History : [. . .] 0 / 2
13:27:26.991 -> Debounce : monitoring
13:27:26.991 -> Inference time : 6057.52 ms
13:27:33.468 -> Raw output : fall=1.6% (sigmoid)
13:27:33.468 ->
```

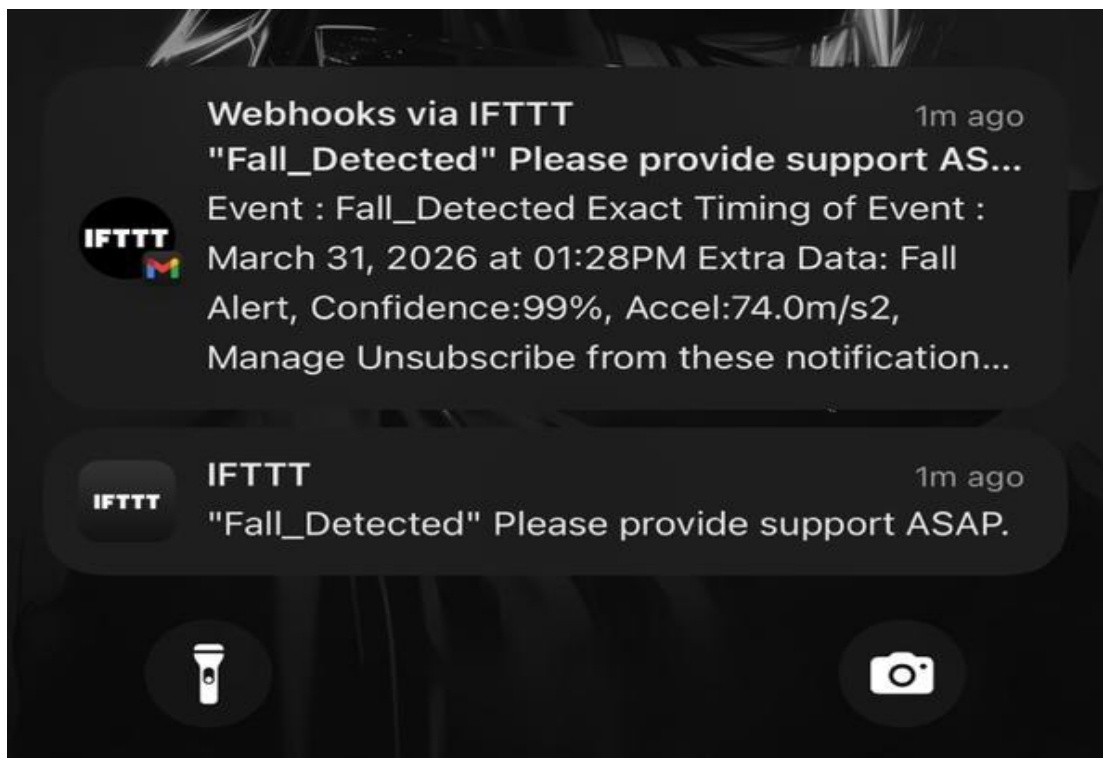

- When fall is detected

```

13:28:11.256 -> [Inference #12 Core 1]
13:28:11.256 -> CNN confidence : 99.6% threshold 60% PASS
13:28:11.256 -> Peak accel : 30.82 m/s2 min 11.0 m/s2 PASS
13:28:11.256 -> Decision : CANDIDATE FALL
13:28:11.256 -> History : [. . o] 1 / 2
13:28:11.256 -> Debounce : monitoring
13:28:11.256 -> Inference time : 6487.58 ms
13:28:17.745 -> Raw output : fall=99.6% (sigmoid)
13:28:17.788 ->
13:28:17.788 -> [Inference #13 Core 1]
13:28:17.788 -> CNN confidence : 99.6% threshold 60% PASS
13:28:17.788 -> Peak accel : 74.03 m/s2 min 11.0 m/s2 PASS
13:28:17.788 -> Decision : CANDIDATE FALL
13:28:17.788 -> History : [o . o] 2 / 2
13:28:17.788 -> Debounce : >>> FALL CONFIRMED <<<
13:28:17.788 -> Inference time : 6486.09 ms
13:28:17.914 ->
13:28:17.914 -> =====
13:28:17.914 -> *** FALL DETECTED ***
13:28:17.914 -> =====
13:28:17.946 -> Event # : 1
13:28:17.946 -> CNN confidence : 99.6%
13:28:17.946 -> Peak accel : 74.03 m/s2
13:28:17.946 -> Debounce : 2/3 windows confirmed
13:28:17.946 -> Timestamp : 90487 ms
13:28:17.946 -> Detection windows:
13:28:17.946 -> Window 1 : 99.6% confidence [FALL]
13:28:17.946 -> Window 2 : 1.2% confidence [normal]
13:28:17.946 -> Window 3 : 99.6% confidence [FALL]
13:28:17.946 -> =====

```

- App and Email Alert Notification



- **ESP32 Pin Diagram**

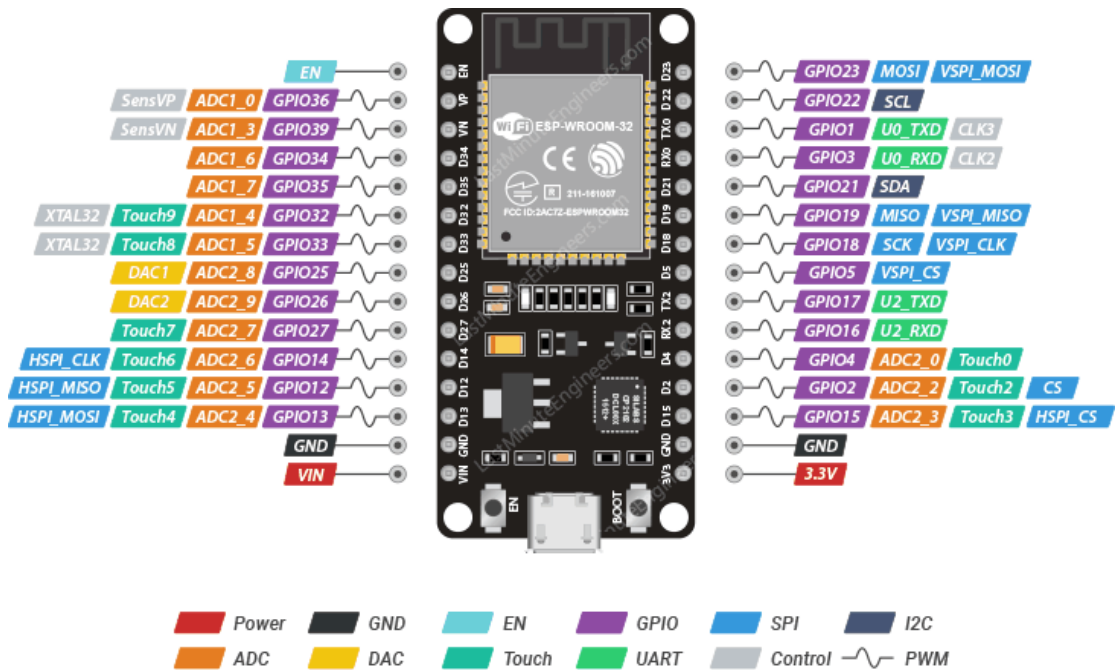


Figure: ESP32 pin diagram

- **MPU 6050 Pin Diagram**

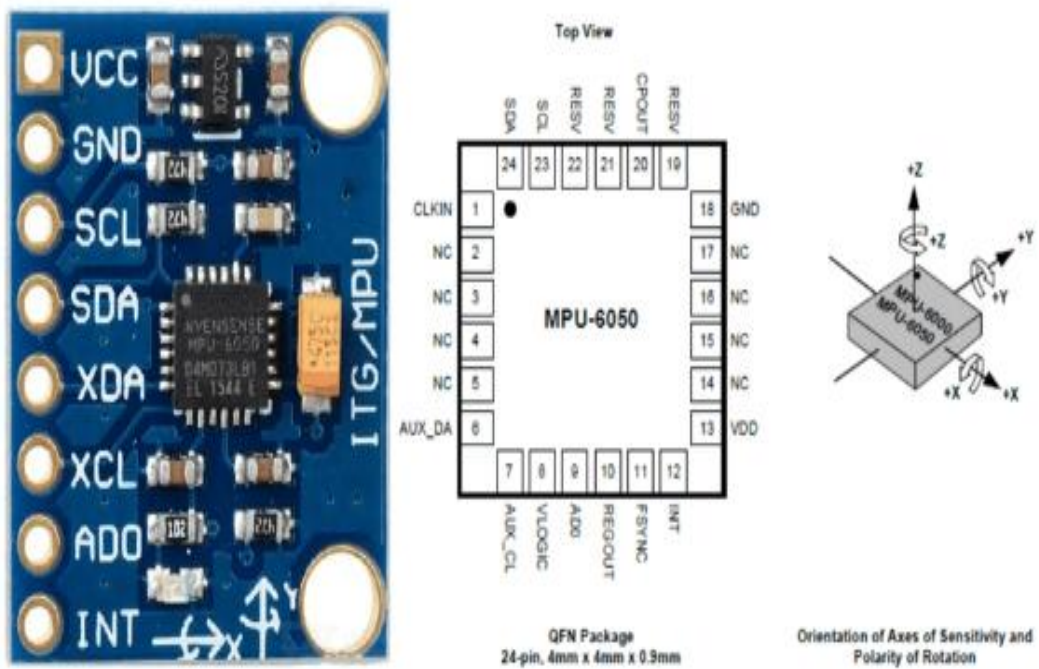


Figure: MPU 6050 pin diagram